



Pontificia Universidad Católica de Chile
Escuela de Ingeniería
Departamento de Ciencia de la Computación

Clase 30: Simulacro Interrogación: LMEval (solución)

Francisco Gazitúa Requena (fco_gazitua_requena@uc.cl)

IIC2113 Diseño Detallado de Software

20 de Noviembre, 2025

Interrogación 2025-1

Un modelo de lenguaje es un tipo de IA que permite predecir la palabra más probable que le sigue a una frase.

Un modelo de lenguaje es un tipo de IA que permite predecir la palabra más probable que le sigue a una frase. Por ejemplo:

- “No he comido y tengo” → “hambre”
- “Raichu es un” → “pokémon”

Un modelo de lenguaje es un tipo de IA que permite predecir la palabra más probable que le sigue a una frase. Por ejemplo:

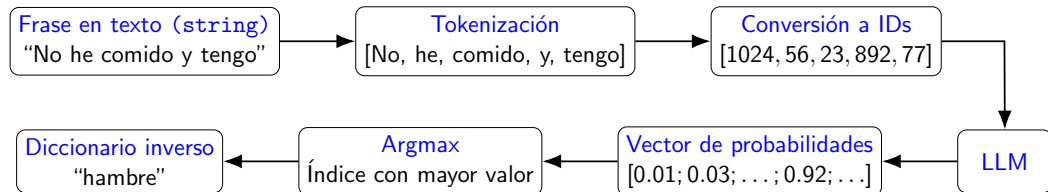
- “No he comido y tengo” → “hambre”
- “Raichu es un” → “pokémon”

Dado que este tipo de IA es entrenada con técnicas de aprendizaje profundo, técnicamente no puede recibir strings, por lo que pasa por el siguiente proceso:

Un modelo de lenguaje es un tipo de IA que permite predecir la palabra más probable que le sigue a una frase. Por ejemplo:

- “No he comido y tengo” → “hambre”
- “Raichu es un” → “pokémon”

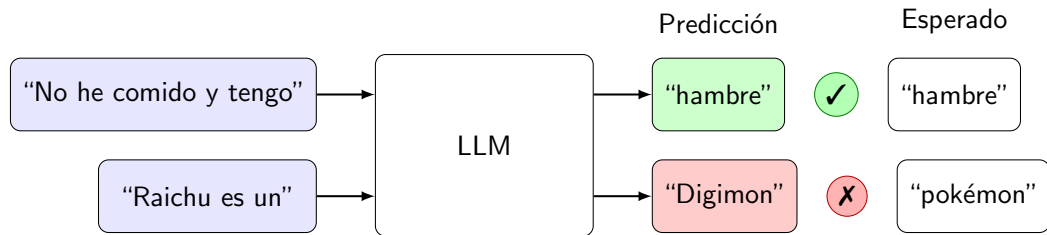
Dado que este tipo de IA es entrenada con técnicas de aprendizaje profundo, técnicamente no puede recibir strings, por lo que pasa por el siguiente proceso:



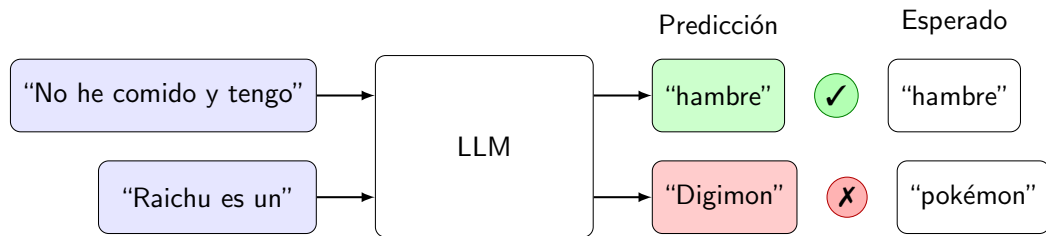
Para evaluar que un modelo de lenguaje funcione correctamente, existen diferentes formas de evaluarlo. Unas de estas se conoce como **next-token-prediction**:

Interrogación 2025-1

Para evaluar que un modelo de lenguaje funcione correctamente, existen diferentes formas de evaluarlo. Unas de estas se conoce como **next-token-prediction**:

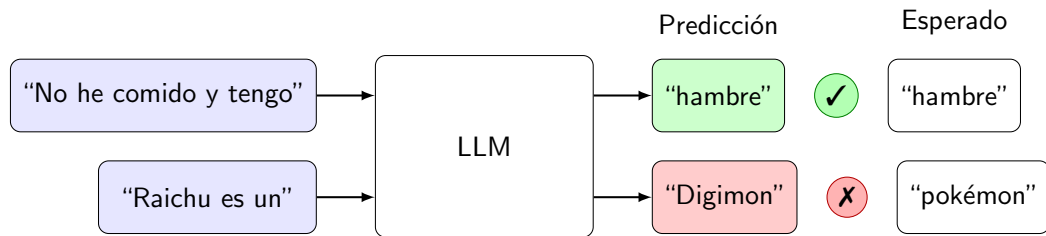


Para evaluar que un modelo de lenguaje funcione correctamente, existen diferentes formas de evaluarlo. Unas de estas se conoce como **next-token-prediction**:



Otro método de evaluación se conoce como **multiple-choice**, el cual funciona similar a **next-token-prediction**, pero chequea que la predicción se encuentre entre un grupo de posibles resultados esperados.

Para evaluar que un modelo de lenguaje funcione correctamente, existen diferentes formas de evaluarlo. Unas de estas se conoce como **next-token-prediction**:



Otro método de evaluación se conoce como **multiple-choice**, el cual funciona similar a **next-token-prediction**, pero chequea que la predicción se encuentre entre un grupo de posibles resultados esperados.

Para ambos métodos, se usa la métrica **precisión** para medir la efectividad del modelo

El proyecto LMEval es una primera versión de un sistema de evaluación de LLMs. Su clase de entrada es `Evaluator.cs` y gestiona la evaluación de dos tipos de Modelos.

El proyecto LMEval es una primera versión de un sistema de evaluación de LLMs. Su clase de entrada es `Evaluator.cs` y gestiona la evaluación de dos tipos de Modelos.

`Evaluator.cs` tiene un método de entrada que recibe un `EvaluationRequest` con información sobre lo que el usuario quiere hacer. En particular:

- Recibe el nombre del modelo que se quiere evaluar, el método de evaluación que se quiere utilizar, el dataset que se quiere usar en la evaluación y, opcionalmente, una ruta a un directorio para guardar el resultado.
- Inicializa un Tokenizador y carga el dataset
- Evalúa al modelo seleccionado con el método de evaluación seleccionado
- Si se especifica, guarda en una ruta el resultado de la evaluación

El proyecto LMEval es una primera versión de un sistema de evaluación de LLMs. Su clase de entrada es `Evaluator.cs` y gestiona la evaluación de dos tipos de Modelos.

`Evaluator.cs` tiene un método de entrada que recibe un `EvaluationRequest` con información sobre lo que el usuario quiere hacer. En particular:

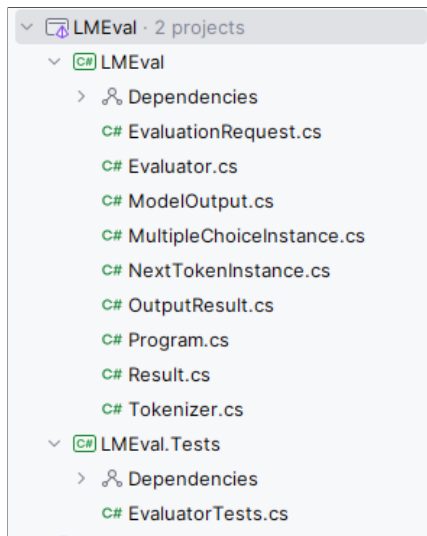
- Recibe el nombre del modelo que se quiere evaluar, el método de evaluación que se quiere utilizar, el dataset que se quiere usar en la evaluación y, opcionalmente, una ruta a un directorio para guardar el resultado.
- Inicializa un Tokenizador y carga el dataset
- Evalúa al modelo seleccionado con el método de evaluación seleccionado
- Si se especifica, guarda en una ruta el resultado de la evaluación

`Evaluate(...)` retorna un objeto de tipo `Result` con información sobre la ejecución de la `EvaluationRequest` (Si la request fue exitosa y el resultado de la evaluación).

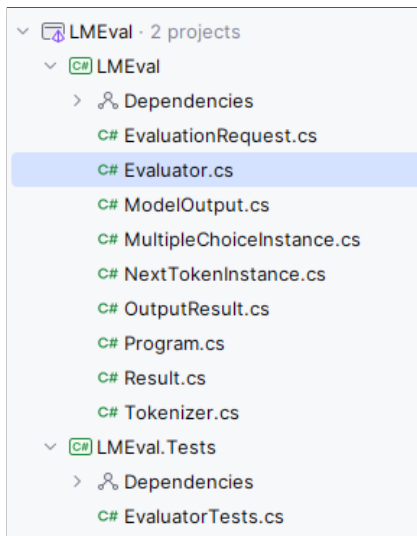
Paso 0: Correr los tests

Paso 1: Entender el Código

Paso 1: Entender el Código



Paso 1: Entender el Código



```
1 public class Evaluator
2 {
3     public int RandomCount = 0;
4     private Tokenizer tokenizer;
5     private List<JsonObject> data;
6     private List<OutputResult> results = new();
7
8     public Result Evaluate(EvaluationRequest request)
9     {
10         // Reset RandomCount and result
11         RandomCount = 0;
12         Result r = new();
13
14         // Try to build the tokenizer
15         string jsonPath = Path.Combine("data", "tokenizers", $"{request.Model}.json");
16         if (!File.Exists(jsonPath))
17         {
18             r.ErrorMessage = "Invalid model type";
19             r.Score = null;
20             return r;
21         }
22         string jsonContent = File.ReadAllText(jsonPath);
23         tokenizer = new Tokenizer(jsonContent);
24
25         // Try load dataset
26         string datasetPath = Path.Combine("data", "datasets", $"{request.Dataset}.json");
27         if (!File.Exists(datasetPath))
28         {
29             r.ErrorMessage = $"Dataset file not found: {request.Dataset}";
30             r.Score = null;
31             return r;
32         }
33     }
34 }
```

```

33     data = JsonSerializer.Deserialize<List<JsonObject>>(File.ReadAllText(datasetPath));
34
35     // Validate request based on EvalType
36     (string, double?) result;
37     switch (request.EvalType)
38     {
39         case "next-token-prediction":
40             result = EvaluateNextTokenPrediction(request.Model, request.OutputDir);
41             break;
42         case "multiple-choice":
43             result = EvaluateMultipleChoice(request.Model, request.OutputDir);
44             break;
45         default:
46             result = ("Invalid evaluation type", null);
47             break;
48     }
49
50     // If outputDir is specified, write results to a file
51     if (request.OutputDir != null) {
52         if (!Directory.Exists(request.OutputDir))
53             result = ("Output directory not found: {request.OutputDir}", null);
54         else
55         {
56             var modelOutput = new ModelOutput(request.Model, request.Dataset, request.EvalType,
57             results, (double)result.Item2!);
58             File.WriteAllText(Path.Combine(request.OutputDir!, "model_output.json"), JsonSerializer.
59             Serialize(modelOutput, new JsonSerializerOptions { WriteIndented = true }));
60         }
61
62         // Set result based on evaluation
63         if (result.Item1 == "Ok")
64         {

```

```

64         r.Score = result.Item2;
65         r.ErrorMessage = null;
66     }
67     else
68     {
69         r.Score = null;
70         r.ErrorMessage = result.Item1;
71     }
72     return r;
73 }
74
75 private (string, double?) EvaluateMultipleChoice(string model, string? outputDir)
76 {
77     // Compute accuracy
78     int succ = 0;
79     results = new();
80     for (int i = 0; i < data.Count; i++)
81     {
82         // Compute OutoutputResult for this data instance
83         MultipleChoiceInstance dataInstance = data[i].Deserialize<MultipleChoiceInstance>();
84         double maxProb = 0.0;
85         int maxIndex = -1;
86         for (int j = 0; j < dataInstance.Choices.Length; j++)
87         {
88             // Encode choice. We need to know each token prob to compute the total choice prob.
89             int[] encodedChoice = tokenizer.Encode(dataInstance.Choices[j]);
90             // Calculate choice probability
91             double choiceProb = 1.0;
92             string auxInput = dataInstance.Input;
93             foreach (var tokenId in encodedChoice)
94             {
95                 double[] probs;

```

```

96         if (model == "random-prev")
97         {
98             probs = GetPrevRandomModelProbs(auxInput);
99         }
100         else if (model == "random")
101         {
102             probs = GetRandomModelProbs();
103         }
104         else
105             return ("Invalid model type", null);
106
107         auxInput += tokenizer.Decode(tokenId); // Append token to input
108         choiceProb *= probs[tokenId]; // Multiply by token probability
109     }
110
111     if (choiceProb > maxProb)
112     {
113         maxProb = choiceProb;
114         maxIndex = j; // Update max index if current choice has higher probability
115     }
116 }
117
118 // Create output result
119 OutputResult result = new();
120 result.Input = dataInstance.Input;
121 result.ExpectedOutput = dataInstance.Choices[dataInstance.CorrectChoiceIndex];
122 result.ActualOutput = dataInstance.Choices[maxIndex];
123 if (result.ActualOutput == result.ExpectedOutput)
124 {
125     succ++;
126     result.IsCorrect = true;
127 }

```

```

128
129     // Add result to the list if outputDir is specified
130     if (outputDir != null)
131         results.Add(result);
132 }
133
134 double score = (double)succ / data.Count;
135
136 return ("Ok", score);
137 }
138
139 private (string, double?) EvaluateNextTokenPrediction(string model, string? outputDir)
140 {
141     // Compute accuracy
142     int succ = 0;
143     results = new();
144     for (int i = 0; i < data.Count; i++)
145     {
146         // Compute OutoutputResult for this data instance
147         NextTokenInstance dataInstance = data[i].Deserialize<NextTokenInstance>(!);
148         // Since the dataset instance contains the input and the expected output concatenated,
149         // we need to split them.
150         string[] parts = dataInstance.Text.Split(" ");
151         var input = string.Join(" ", parts[..^1]); // All but the last part
152         input += " "; // Add a space to separate from the next token
153         var expectedOutput = parts[^1]; // The last part is the expected output
154
155         // Encode expected output. We need to know how many tokens has to generate the model.
156         int[] encodedExpectedOutput = tokenizer.Encode(expectedOutput);
157
158         string auxInput = input;
159         for (int j = 0; j < encodedExpectedOutput.Length; j++)

```

```

160     {
161         double[] probs;
162         if (model == "random")
163             probs = GetRandomModelProbs();
164         else if (model == "random-prev")
165             probs = GetPrevRandomModelProbs(auxInput);
166         else
167             return ("Invalid model type", null);
168
169         // Find the token with the highest probability
170         int maxIdx = 0;
171         for (int k = 1; k < probs.Length; k++)
172         {
173             if (probs[k] > probs[maxIdx])
174                 maxIdx = k;
175         }
176         auxInput += tokenizer.Decode(maxIdx); // Append token to input
177     }
178
179     // Create output result
180     OutputResult result = new();
181     result.Input = input;
182     result.ExpectedOutput = expectedOutput;
183     result.ActualOutput = auxInput[input.Length..]; // Get the generated output after the input
184     if (result.ActualOutput == result.ExpectedOutput)
185     {
186         succ++;
187         result.IsCorrect = true;
188     }
189
190     // Add result to the list if outputDir is specified
191     if (outputDir != null)

```

```

192         results.Add(result);
193     }
194     double score = (double)succ / data.Count;
195
196     return ("Ok", score);
197 }
198
199 private double[] GetPrevRandomModelProbs(string input)
200 {
201     int[] encodedInput = tokenizer.Encode(input);
202     int randomIndex = encodedInput[RandomCount % encodedInput.Length];
203     double[] probabilities =[.. Enumerable.Repeat(1.0, tokenizer.VocabularySize)];
204     probabilities[randomIndex] = tokenizer.VocabularySize;
205     double sum = probabilities.Sum();
206     probabilities =[.. probabilities.Select(p => p / sum)];
207     RandomCount++;
208     return probabilities;
209 }
210
211 private double[] GetRandomModelProbs()
212 {
213     int randomIndex = RandomCount % tokenizer.VocabularySize;
214     double[] probabilities =[.. Enumerable.Repeat(1.0, tokenizer.VocabularySize)];
215     probabilities[randomIndex] = tokenizer.VocabularySize;
216     double sum = probabilities.Sum();
217     probabilities =[.. probabilities.Select(p => p / sum)];
218     RandomCount++;
219     return probabilities;
220 }
221 }

```


Paso 1: Entender el Código

Importante: No es necesario entender **todo** el código para limpiar.

Paso 1: Entender el Código

Importante: No es necesario entender **todo** el código para limpiar.

... veamos lo que hace el código a grandes rasgos.

Paso 1: Entender el Código

Importante: No es necesario entender **todo** el código para limpiar.

... veamos lo que hace el código a grandes rasgos.

Primero, se instancia un Tokenizador:

```
1 string jsonPath = Path.Combine("data", "tokenizers", $"{request.Model}.json");
2 if (!File.Exists(jsonPath))
3 {
4     r.ErrorMessage = "Invalid model type";
5     r.Score = null;
6     return r;
7 }
8 string jsonContent = File.ReadAllText(jsonPath);
9 tokenizer = new Tokenizer(jsonContent);
```

Paso 1: Entender el Código

Luego, cargamos un Dataset:

```
1 string datasetPath = Path.Combine("data", "datasets", $"{request.Dataset}.json");
2 if (!File.Exists(datasetPath))
3 {
4     r.ErrorMessage = $"Dataset file not found: {request.Dataset}";
5     r.Score = null;
6     return r;
7 }
8 data = JsonSerializer.Deserialize<List<JsonObject>>(File.ReadAllText(
    datasetPath));
```

Paso 1: Entender el Código

Después llamamos al método de evaluación

```
1 (string, double?) result;  
2 switch (request.EvalType)  
3 {  
4     case "next-token-prediction":  
5         result = EvaluateNextTokenPrediction(request.Model, request.OutputDir)  
6         ;  
7         break;  
8     case "multiple-choice":  
9         result = EvaluateMultipleChoice(request.Model, request.OutputDir);  
10        break;  
11    default:  
12        result = ("Invalid evaluation type", null);  
13        break;  
14 }
```

Paso 1: Entender el Código

El método de evaluación hace un llamado a un modelo

```
1 private (string, double?) EvaluateNextTokenPrediction(string model, string?
   outputDir)
2 {
3     /* Harta lógica */
4     double[] probs;
5     if (model == "random-prev")
6     {
7         probs = GetPrevRandomModelProbs(auxInput);
8     }
9     else if (model == "random")
10    {
11        probs = GetRandomModelProbs();
12    }
13    else
14        return ("Invalid model type", null);
15    /* Más lógica */
16
17    double score = (double)succ / data.Count;
18
19    return ("Ok", score);
20 }
```

Paso 1: Entender el Código

Luego guardamos el resultado en un directorio si corresponde.

```
1 if (request.OutputDir != null) {  
2     if (!Directory.Exists(request.OutputDir))  
3         result = ($"Output directory not found: {request.OutputDir}", null);  
4     else  
5     {  
6         var modelOutput = new ModelOutput(request.Model, request.Dataset,  
7         request.EvalType, results, (double)result.Item2!);  
8         File.WriteAllText(Path.Combine(request.OutputDir!, "model_output.json"  
9         ), JsonSerializer.Serialize(modelOutput, new JsonSerializerOptions {  
10         WriteIndented = true }));  
11     }  
12 }
```

Paso 1: Entender el Código

Finalmente, creamos un Result.

```
1  if (result.Item1 == "Ok")
2  {
3      r.Score = result.Item2;
4      r.ErrorMessage = null;
5  }
6  else
7  {
8      r.Score = null;
9      r.ErrorMessage = result.Item1;
10 }
11 return r;
```


Paso 1: Entender el Código

El siguiente paso es **detectar un problema importante** en el código.

Paso 1: Entender el Código

El siguiente paso es **detectar un problema importante** en el código.

Si nos fijamos en todo el flujo anterior, tenemos mucho manejo de error:

```
1 string jsonPath = Path.Combine("data", "tokenizers", $"{request.Model}.json");
2 if (!File.Exists(jsonPath))
3 {
4     r.ErrorMessage = "Invalid model type";
5     r.Score = null;
6     return r;
7 }
8 string jsonContent = File.ReadAllText(jsonPath);
9 tokenizer = new Tokenizer(jsonContent);
```

Paso 1: Entender el Código

El siguiente paso es **detectar un problema importante** en el código.

Si nos fijamos en todo el flujo anterior, tenemos mucho manejo de error:

```
1 string jsonPath = Path.Combine("data", "tokenizers", $"{request.Model}.json");
2 if (!File.Exists(jsonPath))
3 {
4     r.ErrorMessage = "Invalid model type";
5     r.Score = null;
6     return r;
7 }
8 string jsonContent = File.ReadAllText(jsonPath);
9 tokenizer = new Tokenizer(jsonContent);
```

... ¿Como podemos arreglar esto?

Paso 1: Entender el Código

Arreglar esto requiere aplicar los principios del cap. 8.

Paso 1: Entender el Código

Arreglar esto requiere aplicar los principios del cap. 8.

1) Hay que encontrar todos los mensajes de error:

```
11 result = ("Invalid evaluation type", null);
```

Paso 1: Entender el Código

Arreglar esto requiere aplicar los principios del cap. 8.

1) Hay que encontrar todos los mensajes de error:

```
11 result = ("Invalid evaluation type", null);
```

2) Tenemos que cambiar todos los mensajes de error por excepciones.

Paso 1: Entender el Código

Arreglar esto requiere aplicar los principios del cap. 8.

1) Hay que encontrar todos los mensajes de error:

```
11 result = ("Invalid evaluation type", null);
```

2) Tenemos que cambiar todos los mensajes de error por excepciones.

3) Dejar de retornar tuplas con el mensaje de error:

```
1 private (string, double?) EvaluateNextTokenPrediction(string model, string?  
    outputDir)
```

Paso 1: Entender el Código

Arreglar esto requiere aplicar los principios del cap. 8.

1) Hay que encontrar todos los mensajes de error:

```
11 result = ("Invalid evaluation type", null);
```

2) Tenemos que cambiar todos los mensajes de error por excepciones.

3) Dejar de retornar tuplas con el mensaje de error:

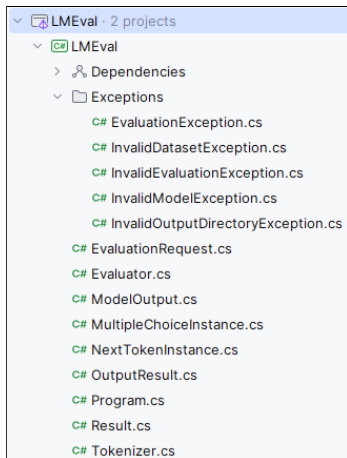
```
1 private (string, double?) EvaluateNextTokenPrediction(string model, string?  
    outputDir)
```

4) Atajar las excepciones en Evaluate y retornar un invalid result en ese caso.

Error Handling

Error Handling

Comencemos creando las excepciones que necesitamos (con sus mensajes de error).



Error Handling

Comencemos creando las excepciones que necesitamos (con sus mensajes de error).

```
1 public abstract class EvaluationException : ApplicationException
2 {
3     public abstract string GetErrorMessage();
4 }
```

Error Handling

Comencemos creando las excepciones que necesitamos (con sus mensajes de error).

```
1 public abstract class EvaluationException : ApplicationException
2 {
3     public abstract string GetErrorMessage();
4 }
```

Luego tenemos una excepción para un dataset inválido.

```
1 public class InvalidDatasetException(string datasetName) : EvaluationException
2 {
3     public override string GetErrorMessage()
4         => $"Dataset file not found: {datasetName}";
5 }
```

Error Handling

Comencemos creando las excepciones que necesitamos (con sus mensajes de error).

```
1 public abstract class EvaluationException : ApplicationException
2 {
3     public abstract string GetErrorMessage();
4 }
```

Luego tenemos una excepción para un dataset inválido.

```
1 public class InvalidDatasetException(string datasetName) : EvaluationException
2 {
3     public override string GetErrorMessage()
4         => $"Dataset file not found: {datasetName}";
5 }
```

También una para un método e evaluación inválido.

```
1 public class InvalidEvaluationException : EvaluationException
2 {
3     public override string GetErrorMessage()
4         => "Invalid evaluation type";
5 }
```

Error Handling

Necesitamos una para cuando el modelo es inválido.

```
1 public class InvalidModelException : EvaluationException
2 {
3     public override string GetErrorMessage()
4         => "Invalid model type";
5 }
```

Error Handling

Necesitamos una para cuando el modelo es inválido.

```
1 public class InvalidModelException : EvaluationException
2 {
3     public override string GetErrorMessage()
4         => "Invalid model type";
5 }
```

Finalmente, necesitamos una para cuando el OutputDir es inválido.

```
1 public class InvalidOutputDirectoryException(string outputDirectory) :
    EvaluationException
2 {
3     public override string GetErrorMessage()
4         => $"Output directory not found: {outputDirectory}";
5 }
```

Error Handling

Ahora utilicemos las excepciones donde corresponda:

Necesitamos una cuando cargamos el Tokenizer

```
1 string jsonPath = Path.Combine("data", "tokenizers", $"{request.Model}.json");  
2 if (!File.Exists(jsonPath))  
3     throw new InvalidModelException();  
4 string jsonContent = File.ReadAllText(jsonPath);  
5 tokenizer = new Tokenizer(jsonContent);
```


Error Handling

Ahora utilizemos las excepciones donde corresponda:

Necesitamos una cuando cargamos el Tokenizer

```
1 string jsonPath = Path.Combine("data", "tokenizers", $"{request.Model}.json");
2 if (!File.Exists(jsonPath))
3     throw new InvalidModelException();
4 string jsonContent = File.ReadAllText(jsonPath);
5 tokenizer = new Tokenizer(jsonContent);
```

Necesitamos otra cuando cargamos el dataset

```
1 string datasetPath = Path.Combine("data", "datasets", $"{request.Dataset}.json");
2 if (!File.Exists(datasetPath))
3     throw new InvalidDatasetException(request.Dataset);
4 data = JsonSerializer.Deserialize<List<JsonObject>>(File.ReadAllText(
    datasetPath));
```

Error Handling

Se necesita una cuando el método de evaluación es inválido

```
1 double result;  
2 switch (request.EvalType)  
3 {  
4     case "next-token-prediction":  
5         result = EvaluateNextTokenPrediction(request.Model, request.OutputDir)  
6         ;  
7         break;  
8     case "multiple-choice":  
9         result = EvaluateMultipleChoice(request.Model, request.OutputDir);  
10        break;  
11    default:  
12        throw new InvalidEvaluationException();  
13 }
```

Error Handling

Se necesita una cuando el método de evaluación es inválido

```
1 double result;  
2 switch (request.EvalType)  
3 {  
4     case "next-token-prediction":  
5         result = EvaluateNextTokenPrediction(request.Model, request.OutputDir)  
6         ;  
7         break;  
8     case "multiple-choice":  
9         result = EvaluateMultipleChoice(request.Model, request.OutputDir);  
10        break;  
11    default:  
12        throw new InvalidEvaluationException();  
13 }
```

Notar que las funciones ya no retornan una tupla

Error Handling

Veamos esto en más detalle

```
1 private double EvaluateNextTokenPrediction(string model, string? outputDir)
2 {
3     /* Harta lógica */
4     double[] probs;
5     if (model == "random-prev")
6     {
7         probs = GetPrevRandomModelProbs(auxInput);
8     }
9     else if (model == "random")
10    {
11        probs = GetRandomModelProbs();
12    }
13    else
14        throw new InvalidModelException();
15    /* Más lógica */
16
17    return (double)succ / data.Count;
18 }
```

Error Handling

Veamos esto en más detalle

```
1 private double EvaluateNextTokenPrediction(string model, string? outputDir)
2 {
3     /* Harta lógica */
4     double[] probs;
5     if (model == "random-prev")
6     {
7         probs = GetPrevRandomModelProbs(auxInput);
8     }
9     else if (model == "random")
10    {
11        probs = GetRandomModelProbs();
12    }
13    else
14        throw new InvalidModelException();
15    /* Más lógica */
16
17    return (double)succ / data.Count;
18 }
```

Esto se logro gracias al manejo de error

Error Handling

Finalmente, necesitamos una al guardar el resultado en el OutputDir

```
1 if (request.OutputDir != null)
2 {
3     if (!Directory.Exists(request.OutputDir))
4         throw new InvalidOutputDirectoryException(request.OutputDir);
5     var modelOutput = new ModelOutput(request.Model, request.Dataset, request.
6     EvalType, results, result);
7     File.WriteAllText(Path.Combine(request.OutputDir!, "model_output.json"),
8     JsonSerializer.Serialize(modelOutput, new JsonSerializerOptions {
9     WriteIndented = true }));
10 }
```

Error Handling

Ahora solo nos hace falta atajar las excepciones

```
1 public class Evaluator
2 {
3     public int RandomCount = 0;
4     private Tokenizer tokenizer;
5     private List<JsonObject> data;
6     private List<OutputResult> results = new();
7     private Result _result = new();
8
9     public Result Evaluate(EvaluationRequest request)
10    {
11        try
12        {
13            _result = new Result();
14            TryToEvaluate(request);
15        }
16        catch (EvaluationException e)
17        {
18            _result.ErrorMessage = e.GetErrorMessage();
19            _result.Score = null;
20        }
21
22        return _result;
23    }
```

Error Handling

```
24
25 private void TryToEvaluate(EvaluationRequest request)
26 {
27     // Reset RandomCount and result
28     RandomCount = 0;
29
30     // Try to build the tokenizer
31     string jsonPath = Path.Combine("data", "tokenizers", $"{request.Model}.json");
32     if (!File.Exists(jsonPath))
33         throw new InvalidModelException();
34     string jsonContent = File.ReadAllText(jsonPath);
35     tokenizer = new Tokenizer(jsonContent);
36
37     // Try load dataset
38     string datasetPath = Path.Combine("data", "datasets", $"{request.Dataset}.json");
39     if (!File.Exists(datasetPath))
40         throw new InvalidDatasetException(request.Dataset);
41     data = JsonSerializer.Deserialize<List<JsonObject>>(File.ReadAllText(datasetPath));
42
43     // Validate request based on EvalType
44     double result;
45     switch (request.EvalType)
46     {
47     case "next-token-prediction":
48         result = EvaluateNextTokenPrediction(request.Model, request.OutputDir);
49         break;
50     case "multiple-choice":
51         result = EvaluateMultipleChoice(request.Model, request.OutputDir);
52         break;
```


Error Handling

```
53         default:
54             throw new InvalidEvaluationException();
55     }
56
57     // If outputDir is specified, write results to a file
58     if (request.OutputDir != null)
59     {
60         if (!Directory.Exists(request.OutputDir))
61             throw new InvalidOutputDirectoryException(request.OutputDir);
62         var modelOutput = new ModelOutput(request.Model, request.Dataset, request.EvalType, results,
        result);
63         File.WriteAllText(Path.Combine(request.OutputDir!, "model_output.json"), JsonSerializer.
        Serialize(modelOutput, new JsonSerializerOptions { WriteIndented = true }));
64     }
65
66     // Set result based on evaluation
67     _result.Score = result;
68     _result.ErrorMessage = null;
69 }
```

Objects vs Data Structures

Objects vs Data Structures

Ahora debemos **encontrar otro problema importante** en el código.

Objects vs Data Structures

Ahora debemos **encontrar otro problema importante** en el código.

Si nos fijamos, la modelación de los modelos rompe el Open-Closed Principle

```
1 private (string, double?) EvaluateNextTokenPrediction(string model, string? outputDir)
2 {
3     /* Harta lógica */
4     double[] probs;
5     if (model == "random-prev")
6     {
7         probs = GetPrevRandomModelProbs(auxInput);
8     }
9     else if (model == "random")
10    {
11        probs = GetRandomModelProbs();
12    }
13    else
14        return ("Invalid model type", null);
15    /* Más lógica */
16
17    double score = (double)succ / data.Count;
18
19    return ("Ok", score);
20 }
```

Objects vs Data Structures

No nos compliquemos y hagamos esto de la forma más trivial posible.

```
1 public interface IModel {  
2     public double[] GenerateModelOutput(string input);  
3 }
```

Objects vs Data Structures

No nos compliquemos y hagamos esto de la forma más trivial posible.

```
1 public interface IModel {  
2     public double[] GenerateModelOutput(string input);  
3 }
```

Luego tenemos una factory para crear el modelo apropiado:

```
1 public static class ModelFactory {  
2     public static IModel Create(string modelName, Tokenizer tokenizer)  
3     {  
4         switch (modelName)  
5         {  
6             case "random":  
7                 return new RandomModel(tokenizer);  
8             case "random-prev":  
9                 return new RandomPrevModel(tokenizer);  
10            default:  
11                throw new InvalidModelException();  
12            }  
13        }  
14    }
```

Objects vs Data Structures

Ahora creamos una clase para cada modelo.

Objects vs Data Structures

Ahora creamos una clase para cada modelo.

RandomModel tendrá el mismo código que el método GetRandomModelProbs()

```
1 public class RandomModel(Tokenizer tokenizer) : IModel
2 {
3     private int _randomCount = 0;
4
5     public double[] GenerateModelOutput(string input)
6     {
7         int randomIndex = _randomCount % tokenizer.VocabularySize;
8         double[] probabilities =[.. Enumerable.Repeat(1.0, tokenizer.
9 VocabularySize)];
10        probabilities[randomIndex] = tokenizer.VocabularySize;
11        double sum = probabilities.Sum();
12        probabilities =[.. probabilities.Select(p => p / sum)];
13        _randomCount++;
14        return probabilities;
15    }
16 }
```


Objects vs Data Structures

Ahora creamos una clase para cada modelo.

RandomPrevModel tendrá el mismo código que el método
GetPrevRandomModelProbs()

```
1 public class RandomPrevModel(Tokenizer tokenizer) : IModel
2 {
3     private int _randomCount = 0;
4
5     public double[] GenerateModelOutput(string input)
6     {
7         int[] encodedInput = tokenizer.Encode(input);
8         int randomIndex = encodedInput[_randomCount % encodedInput.Length];
9         double[] probabilities =[.. Enumerable.Repeat(1.0, tokenizer.
VocabularySize)];
10         probabilities[randomIndex] = tokenizer.VocabularySize;
11         double sum = probabilities.Sum();
12         probabilities =[.. probabilities.Select(p => p / sum)];
13         _randomCount++;
14         return probabilities;
15     }
16 }
```

Objects vs Data Structures

Luego, debemos utilizar la factory y crear el modelo:

Podemos pasar el modelo directamente a las funciones con el método de evaluación

```
1 IModel model = ModelFactory.Create(request.Model, tokenizer);
2
3 double result;
4 switch (request.EvalType)
5 {
6     case "next-token-prediction":
7         result = EvaluateNextTokenPrediction(model, request.OutputDir);
8         break;
9     case "multiple-choice":
10        result = EvaluateMultipleChoice(model, request.OutputDir);
11        break;
12    default:
13        throw new InvalidEvaluationException();
14 }
```

Objects vs Data Structures

Luego, debemos utilizar la factory y crear el modelo:

Luego, el código que utiliza el modelo queda bastante simple

```
1 private double EvaluateNextTokenPrediction(IModel model, string? outputDir)
2 {
3     /* Harta lógica */
4     double[] probs = model.GenerateModelOutput(auxInput);
5     /* Más lógica */
6
7     return (double)succ / data.Count;
8 }
```

Por el momento, hemos solucionado 2 problemas pequeños, pero importantes...

Objects vs Data Structures

Luego, debemos utilizar la factory y crear el modelo:

Luego, el código que utiliza el modelo queda bastante simple

```
1 private double EvaluateNextTokenPrediction(IModel model, string? outputDir)
2 {
3     /* Harta lógica */
4     double[] probs = model.GenerateModelOutput(auxInput);
5     /* Más lógica */
6
7     return (double)succ / data.Count;
8 }
```

Por el momento, hemos solucionado 2 problemas pequeños, pero importantes...

... Pero el código aún sigue bastante sucio

Paso 1: Entender el Código

Paso 1: Entender el Código

Ya entendimos el flujo general del programa, pero nos hace falta entender como funcionan exactamente los métodos de evaluación.

Paso 1: Entender el Código

Ya entendimos el flujo general del programa, pero nos hace falta entender como funcionan exactamente los métodos de evaluación.

De forma general, estos siguen los siguientes pasos:

- 1 Recorre cada dato del dataset

Paso 1: Entender el Código

Ya entendimos el flujo general del programa, pero nos hace falta entender como funcionan exactamente los métodos de evaluación.

De forma general, estos siguen los siguientes pasos:

- 1 Recorre cada dato del dataset
- 2 Utilizando el método de evaluación correspondiente, genera un `OutputResult` para cada dato del dataset

Paso 1: Entender el Código

Ya entendimos el flujo general del programa, pero nos hace falta entender como funcionan exactamente los métodos de evaluación.

De forma general, estos siguen los siguientes pasos:

- 1 Recorre cada dato del dataset
- 2 Utilizando el método de evaluación correspondiente, genera un `OutputResult` para cada dato del dataset
- 3 Por cada `OutputResult` generado, actualiza el contador de aciertos

Paso 1: Entender el Código

Ya entendimos el flujo general del programa, pero nos hace falta entender como funcionan exactamente los métodos de evaluación.

De forma general, estos siguen los siguientes pasos:

- 1 Recorre cada dato del dataset
- 2 Utilizando el método de evaluación correspondiente, genera un `OutputResult` para cada dato del dataset
- 3 Por cada `OutputResult` generado, actualiza el contador de aciertos
- 4 Añade cada `OutputResult` a una colección

Paso 1: Entender el Código

Ya entendimos el flujo general del programa, pero nos hace falta entender como funcionan exactamente los métodos de evaluación.

De forma general, estos siguen los siguientes pasos:

- 1 Recorre cada dato del dataset
- 2 Utilizando el método de evaluación correspondiente, genera un `OutputResult` para cada dato del dataset
- 3 Por cada `OutputResult` generado, actualiza el contador de aciertos
- 4 Añade cada `OutputResult` a una colección

Veamos esto en detalle

```

1 private double EvaluateMultipleChoice(IModel model, string? outputDir)
2 {
3     // Compute accuracy
4     int succ = 0;
5     results = new();
6     for (int i = 0; i < data.Count; i++)
7     {
8         // Compute OutoutputResult for this data instance
9         MultipleChoiceInstance dataInstance = data[i].Deserialize<MultipleChoiceInstance>(!);
10        double maxProb = 0.0;
11        int maxIndex = -1;
12        for (int j = 0; j < dataInstance.Choices.Length; j++)
13        {
14            // Encode choice. We need to know each token prob to compute the total choice prob.
15            int[] encodedChoice = tokenizer.Encode(dataInstance.Choices[j]);
16            // Calculate choice probability
17            double choiceProb = 1.0;
18            string auxInput = dataInstance.Input;
19            foreach (var tokenId in encodedChoice)
20            {
21                double[] probs = model.GenerateModelOutput(auxInput);
22
23                auxInput += tokenizer.Decode(tokenId); // Append token to input
24                choiceProb *= probs[tokenId]; // Multiply by token probability
25            }
26
27            if (choiceProb > maxProb)
28            {
29                maxProb = choiceProb;
30                maxIndex = j; // Update max index if current choice has higher probability
31            }
32        }
33    }

```

```
34     // Create output result
35     OutputResult result = new();
36     result.Input = dataInstance.Input;
37     result.ExpectedOutput = dataInstance.Choices[dataInstance.CorrectChoiceIndex];
38     result.ActualOutput = dataInstance.Choices[maxIndex];
39     if (result.ActualOutput == result.ExpectedOutput)
40     {
41         succ++;
42         result.IsCorrect = true;
43     }
44
45     // Add result to the list if outputDir is specified
46     if (outputDir != null)
47         results.Add(result);
48 }
49
50 double score = (double)succ / data.Count;
51
52 return score;
53 }
```

```

1 private double EvaluateNextTokenPrediction(IModel model, string? outputDir)
2 {
3     // Compute accuracy
4     int succ = 0;
5     results = new();
6     for (int i = 0; i < data.Count; i++)
7     {
8         // Compute OutoutputResult for this data instance
9         NextTokenInstance dataInstance = data[i].Deserialize<NextTokenInstance>(!);
10        // Since the dataset instance contains the input and the expected output concatenated,
11        // we need to split them.
12        string[] parts = dataInstance.Text.Split(" ");
13        var input = string.Join(" ", parts[..^1]); // All but the last part
14        input += " "; // Add a space to separate from the next token
15        var expectedOutput = parts[^1]; // The last part is the expected output
16
17        // Encode expected output. We need to know how many tokens has to generate the model.
18        int[] encodedExpectedOutput = tokenizer.Encode(expectedOutput);
19
20        string auxInput = input;
21        for (int j = 0; j < encodedExpectedOutput.Length; j++)
22        {
23            double[] probs = model.GenerateModelOutput(auxInput);
24
25            // Find the token with the highest probability
26            int maxIdx = 0;
27            for (int k = 1; k < probs.Length; k++)
28            {
29                if (probs[k] > probs[maxIdx])
30                    maxIdx = k;
31            }
32            auxInput += tokenizer.Decode(maxIdx); // Append token to input
33        }

```

```
34
35 // Create output result
36 OutputResult result = new();
37 result.Input = input;
38 result.ExpectedOutput = expectedOutput;
39 result.ActualOutput = auxInput[input.Length..]; // Get the generated output after the input
40 if (result.ActualOutput == result.ExpectedOutput)
41 {
42     succ++;
43     result.IsCorrect = true;
44 }
45
46 // Add result to the list if outputDir is specified
47 if (outputDir != null)
48     results.Add(result);
49 }
50 double score = (double)succ / data.Count;
51
52 return score;
53 }
```

Objects vs Data Structures

Objects vs Data Structures

Como vimos, los métodos de evaluación cambian en solo un paso, por lo que podemos utilizar un **Template**

Objects vs Data Structures

Como vimos, los métodos de evaluación cambian en solo un paso, por lo que podemos utilizar un **Template**

```
1 public abstract class EvaluationMethod(List<JsonObject> dataset){
2     private int _successes = 0;
3
4     public List<OutputResult> GenerateModelResults() {
5         List<OutputResult> results = new();
6         for(int i = 0; i < dataset.Count; i++)
7         {
8             OutputResult result = GenerateInstanceResult(dataset[i]);
9             results.Add(result);
10            UpdateSuccesses(result);
11        }
12
13        return results;
14    }
15
16    protected abstract OutputResult GenerateInstanceResult(JsonObject instance);
17
18    private void UpdateSuccesses(OutputResult result) {
19        if (result.ActualOutput == result.ExpectedOutput)
20        {
21            _successes++;
22            result.IsCorrect = true;
23        }
24    }
25 }
```

Objects vs Data Structures

De este template heredaran clases abstractas. Para crearlas necesitamos una factory

```
1 public static class EvaluationMethodFactory
2 {
3     public static EvaluationMethod Create(string evaluationType, List<
4     JsonObject> dataset, Tokenizer tokenizer, IModel model)
5     {
6         switch (evaluationType)
7         {
8             case "next-token-prediction":
9                 return new NextTokenPrediction(dataset, tokenizer, model);
10            case "multiple-choice":
11                return new MultipleChoice(dataset, tokenizer, model);
12            default:
13                throw new InvalidEvaluationException();
14        }
15    }
16 }
```

Objects vs Data Structures

De este template heredaran clases abstractas. Para crearlas necesitamos una factory

```
1 public static class EvaluationMethodFactory
2 {
3     public static EvaluationMethod Create(string evaluationType, List<
4     JsonObject> dataset, Tokenizer tokenizer, IModel model)
5     {
6         switch (evaluationType)
7         {
8             case "next-token-prediction":
9                 return new NextTokenPrediction(dataset, tokenizer, model);
10            case "multiple-choice":
11                return new MultipleChoice(dataset, tokenizer, model);
12            default:
13                throw new InvalidEvaluationException();
14        }
15    }
16 }
```

Veamos como quedan los métodos de evaluación

```

1 public class MultipleChoice(List<JsonObject> dataset, Tokenizer tokenizer, IModel model):
    EvaluationMethod(dataset){
2     private OutputResult _result = new ();
3     private double _maxProbability = 0.0;
4     private int _maxIndex = -1;
5     protected override OutputResult GenerateInstanceResult(JsonObject instance) {
6         MultipleChoiceInstance dataInstance = instance.Deserialize<MultipleChoiceInstance>(!);
7         InitializeFrom(dataInstance);
8         ComputeMaxIndex(dataInstance);
9         _result.ActualOutput = dataInstance.Choices[_maxIndex];
10        return _result;
11    }
12    private void InitializeFrom(MultipleChoiceInstance instance) {
13        _maxProbability = 0.0;
14        _maxIndex = -1;
15        _result = new();
16        _result.Input = instance.Input;
17        _result.ExpectedOutput = instance.Choices[instance.CorrectChoiceIndex];
18    }
19    private void ComputeMaxIndex(MultipleChoiceInstance instance) {
20        for (int j = 0; j < instance.Choices.Length; j++){
21            int[] encodedChoice = tokenizer.Encode(instance.Choices[j]);
22            double choiceProb = 1.0;
23            string auxInput = instance.Input;
24            foreach (var tokenId in encodedChoice){
25                double[] probs = model.GenerateModelOutput(auxInput);
26                auxInput += tokenizer.Decode(tokenId);
27                choiceProb *= probs[tokenId];
28            }
29            if (choiceProb > _maxProbability){
30                _maxProbability = choiceProb;
31                _maxIndex = j;
32            }
33        }
34    }

```

```

1 public class NextTokenPrediction(List<JsonObject> dataset, Tokenizer tokenizer, IModel model) :
2     EvaluationMethod(dataset) {
3     private OutputResult _result = new ();
4     protected override OutputResult GenerateInstanceResult(JsonObject instance) {
5         NextTokenInstance dataInstance = instance.Deserialize<NextTokenInstance>(!);
6         InitializeFrom(dataInstance);
7         GenerateOutput();
8         return _result;
9     }
10    private void InitializeFrom(NextTokenInstance instance) {
11        _result = new OutputResult();
12        string[] parts = instance.Text.Split(" ");
13        _result.Input = string.Join(" ", parts[..^1]) + " ";
14        _result.ExpectedOutput = parts[^1];
15    }
16    private void GenerateOutput() {
17        string auxInput = _result.Input;
18        int[] encodedExpectedOutput = tokenizer.Encode(_result.ExpectedOutput);
19        for (int j = 0; j < encodedExpectedOutput.Length; j++)
20        {
21            double[] probs = model.GenerateModelOutput(auxInput);
22            int maxIdx = 0;
23            for (int k = 1; k < probs.Length; k++)
24            {
25                if (probs[k] > probs[maxIdx])
26                    maxIdx = k;
27            }
28            auxInput += tokenizer.Decode(maxIdx);
29        }
30        _result.ActualOutput = auxInput[_result.Input.Length..];
31    }
32 }

```

```
1 public class Evaluator
2 {
3     private Tokenizer tokenizer;
4     private List<JsonObject> data;
5     private Result _result = new();
6
7     public Result Evaluate(EvaluationRequest request)
8     {
9         try
10         {
11             _result = new Result();
12             TryToEvaluate(request);
13         }
14         catch (EvaluationException e)
15         {
16             _result.ErrorMessage = e.GetErrorMessage();
17             _result.Score = null;
18         }
19
20         return _result;
21     }
22
23     private void TryToEvaluate(EvaluationRequest request)
24     {
25         // Try to build the tokenizer
26         string jsonPath = Path.Combine("data", "tokenizers", $"{request.Model}.json");
27         if (!File.Exists(jsonPath))
28             throw new InvalidModelException();
29         string jsonContent = File.ReadAllText(jsonPath);
30         tokenizer = new Tokenizer(jsonContent);
31
32         // Try load dataset
33         string datasetPath = Path.Combine("data", "datasets", $"{request.Dataset}.json");
```

```

34     if (!File.Exists(datasetPath))
35         throw new InvalidDatasetException(request.Dataset);
36     data = JsonSerializer.Deserialize<List<JsonObject>>(File.ReadAllText(datasetPath));
37
38     IModel model = ModelFactory.Create(request.Model, tokenizer);
39
40     EvaluationMethod method = EvaluationMethodFactory.Create(request.EvalType, data, tokenizer,
41 model);
42
43     var results = method.GenerateModelResults();
44     int sucessess = 0;
45     foreach (var res in results)
46     {
47         if (res.IsCorrect)
48             sucessess++;
49     }
50
51     double result = (double)sucessess / data.Count;
52
53     // If outputDir is specified, write results to a file
54     if (request.OutputDir != null)
55     {
56         if (!Directory.Exists(request.OutputDir))
57             throw new InvalidOutputDirectoryException(request.OutputDir);
58         var modelOutput = new ModelOutput(request.Model, request.Dataset, request.EvalType, results,
59 result);
60         File.WriteAllText(Path.Combine(request.OutputDir!, "model_output.json"), JsonSerializer.
61 Serialize(modelOutput, new JsonSerializerOptions { WriteIndented = true }));
62     }
63
64     // Set result based on evaluation
65     _result.Score = result;
66     _result.ErrorMessage = null;
67 }
68 }

```


Objects vs Data Structures

Ya estamos cerca de terminar, ya que solucionamos los problemas más importantes de nuestro código. Sin embargo, aún nos quedan algunos problemas que resolver en la clase Evaluator.

Objects vs Data Structures

Ya estamos cerca de terminar, ya que solucionamos los problemas más importantes de nuestro código. Sin embargo, aún nos quedan algunos problemas que resolver en la clase `Evaluator`.

Como acabamos de ver, el método `TryToEvaluate()` está bastante largo. Tiene las siguientes responsabilidades:

Objects vs Data Structures

Ya estamos cerca de terminar, ya que solucionamos los problemas más importantes de nuestro código. Sin embargo, aún nos quedan algunos problemas que resolver en la clase `Evaluator`.

Como acabamos de ver, el método `TryToEvaluate()` está bastante largo. Tiene las siguientes responsabilidades:

- Inicializa el `Tokenizer`

Objects vs Data Structures

Ya estamos cerca de terminar, ya que solucionamos los problemas más importantes de nuestro código. Sin embargo, aún nos quedan algunos problemas que resolver en la clase `Evaluator`.

Como acabamos de ver, el método `TryToEvaluate()` está bastante largo. Tiene las siguientes responsabilidades:

- Inicializa el `Tokenizer`
- Carga el dataset

Objects vs Data Structures

Ya estamos cerca de terminar, ya que solucionamos los problemas más importantes de nuestro código. Sin embargo, aún nos quedan algunos problemas que resolver en la clase `Evaluator`.

Como acabamos de ver, el método `TryToEvaluate()` está bastante largo. Tiene las siguientes responsabilidades:

- Inicializa el `Tokenizer`
- Carga el dataset
- Calcula el accuracy

Objects vs Data Structures

Ya estamos cerca de terminar, ya que solucionamos los problemas más importantes de nuestro código. Sin embargo, aún nos quedan algunos problemas que resolver en la clase `Evaluator`.

Como acabamos de ver, el método `TryToEvaluate()` está bastante largo. Tiene las siguientes responsabilidades:

- Inicializa el `Tokenizer`
- Carga el dataset
- Calcula el accuracy
- Carga los resultados en el `OutputDir`

Objects vs Data Structures

Ya estamos cerca de terminar, ya que solucionamos los problemas más importantes de nuestro código. Sin embargo, aún nos quedan algunos problemas que resolver en la clase `Evaluator`.

Como acabamos de ver, el método `TryToEvaluate()` está bastante largo. Tiene las siguientes responsabilidades:

- Inicializa el `Tokenizer`
- Carga el dataset
- Calcula el accuracy
- Carga los resultados en el `OutputDir`
- Maneja el flujo principal del programa

Objects vs Data Structures

Ya estamos cerca de terminar, ya que solucionamos los problemas más importantes de nuestro código. Sin embargo, aún nos quedan algunos problemas que resolver en la clase `Evaluator`.

Como acabamos de ver, el método `TryToEvaluate()` está bastante largo. Tiene las siguientes responsabilidades:

- Inicializa el `Tokenizer`
- Carga el dataset
- Calcula el accuracy
- Carga los resultados en el `OutputDir`
- Maneja el flujo principal del programa

Estas son muchas responsabilidades para un simple controlador.

Boundaries 🤝 Classes

El problema principal es que la clase `Evaluator` está haciendo muchas cosas. Tenemos que separar sus responsabilidades en otras clases. Para esto debemos aplicar principios de los capítulos 8 y 10.

El problema principal es que la clase `Evaluator` está haciendo muchas cosas. Tenemos que separar sus responsabilidades en otras clases. Para esto debemos aplicar principios de los capítulos 8 y 10.

Tenemos que:

- Delegar la creación del `Tokenizer` a otra clase

El problema principal es que la clase `Evaluator` está haciendo muchas cosas. Tenemos que separar sus responsabilidades en otras clases. Para esto debemos aplicar principios de los capítulos 8 y 10.

Tenemos que:

- Delegar la creación del `Tokenizer` a otra clase
- Encapsular el dataset

El problema principal es que la clase `Evaluator` está haciendo muchas cosas. Tenemos que separar sus responsabilidades en otras clases. Para esto debemos aplicar principios de los capítulos 8 y 10.

Tenemos que:

- Delegar la creación del `Tokenizer` a otra clase
- Encapsular el dataset
- Encapsular la colección de `OutputResult`

El problema principal es que la clase `Evaluator` está haciendo muchas cosas. Tenemos que separar sus responsabilidades en otras clases. Para esto debemos aplicar principios de los capítulos 8 y 10.

Tenemos que:

- Delegar la creación del `Tokenizer` a otra clase
- Encapsular el dataset
- Encapsular la colección de `OutputResult`
- Delegar la carga al `OutputDir` a otra clase

El problema principal es que la clase `Evaluator` está haciendo muchas cosas. Tenemos que separar sus responsabilidades en otras clases. Para esto debemos aplicar principios de los capítulos 8 y 10.

Tenemos que:

- Delegar la creación del `Tokenizer` a otra clase
- Encapsular el dataset
- Encapsular la colección de `OutputResult`
- Delegar la carga al `OutputDir` a otra clase

comencemos con el `Tokenizer`

```
1 public class Tokenizer
2 {
3     private readonly Dictionary<string, int> tokenizer;
4     private readonly Dictionary<int, string> reverseTokenizer;
5
6     public Tokenizer(string content)
7     {
8         tokenizer = JsonSerializer.Deserialize<Dictionary<string, int>>(content)!;
9         reverseTokenizer = tokenizer.ToDictionary(kvp => kvp.Value, kvp => kvp.Key);
10    }
11
12    public int VocabularySize => tokenizer.Count;
13
14    public int[] Encode(string text)
15    {
16        var encoded = new List<int>();
17        while (text.Length > 0)
18        {
19            string token = GetLongestMatchingToken(text);
20            encoded.Add(tokenizer[token]);
21            text = text[token.Length..];
22        }
23        return[.. encoded];
24    }
25
26    private string GetLongestMatchingToken(string token)
27    {
28        while (!tokenizer.ContainsKey(token))
29            token = token[..^1];
30        return token;
31    }
32
33    public string Decode(int token)
34        => reverseTokenizer[token];
35 }
```


Podemos simplemente delegar toda la creación del Tokenizer al constructor de la clase Tokenizer:

```
1 public Tokenizer(string modelName) {  
2     string jsonPath = Path.Combine("data", "tokenizers", $"{modelName}.json");  
3     if (!File.Exists(jsonPath))  
4         throw new InvalidModelException();  
5     string content = File.ReadAllText(jsonPath);  
6     tokenizer = JsonSerializer.Deserialize<Dictionary<string, int>>(content)!;  
7     reverseTokenizer = tokenizer.ToDictionary(kvp => kvp.Value, kvp => kvp.Key  
8 );  
9 }
```

Boundries & Classes

Podemos simplemente delegar toda la creación del Tokenizer al constructor de la clase Tokenizer:

```
1 public Tokenizer(string modelName) {  
2     string jsonPath = Path.Combine("data", "tokenizers", $"{modelName}.json");  
3     if (!File.Exists(jsonPath))  
4         throw new InvalidModelException();  
5     string content = File.ReadAllText(jsonPath);  
6     tokenizer = JsonSerializer.Deserialize<Dictionary<string, int>>(content)!;  
7     reverseTokenizer = tokenizer.ToDictionary(kvp => kvp.Value, kvp => kvp.Key  
8 );  
9 }
```

Ahora veamos el dataset

Estamos cargando el dataset a partir de un json y estamos deserializandolo en una `List<JsonObject>`:

```
1 string datasetPath = Path.Combine("data", "datasets", $"{request.Dataset}.json");
2 if (!File.Exists(datasetPath))
3     throw new InvalidDatasetException(request.Dataset);
4 data = JsonSerializer.Deserialize<List<JsonObject>>(File.ReadAllText(
    datasetPath));
```

Boundries & Classes

Estamos cargando el dataset a partir de un json y estamos deserializandolo en una `List<JsonObject>`:

```
1 string datasetPath = Path.Combine("data", "datasets", $"{request.Dataset}.json");
2 if (!File.Exists(datasetPath))
3     throw new InvalidDatasetException(request.Dataset);
4 data = JsonSerializer.Deserialize<List<JsonObject>>(File.ReadAllText(
    datasetPath));
```

Tenemos que encapsular esto en su propia clase

Boundries & Classes

```
1 public class Dataset
2 {
3     private List<JsonObject> _data;
4
5     public int Count => _data.Count;
6
7     public Dataset(string datasetName)
8     {
9         string datasetPath = Path.Combine("data", "datasets", $"{datasetName}.
10 json");
11         if (!File.Exists(datasetPath))
12             throw new InvalidDatasetException(datasetName);
13         _data = JsonSerializer.Deserialize<List<JsonObject>>(File.ReadAllText(
14 datasetPath));
15     }
16
17     public JsonObject GetByIndex(int index)
18         => _data[index];
19 }
```

Boundries & Classes

```
1 public class Dataset
2 {
3     private List<JsonObject> _data;
4
5     public int Count => _data.Count;
6
7     public Dataset(string datasetName)
8     {
9         string datasetPath = Path.Combine("data", "datasets", $"{datasetName}.
10 json");
11         if (!File.Exists(datasetPath))
12             throw new InvalidDatasetException(datasetName);
13         _data = JsonSerializer.Deserialize<List<JsonObject>>(File.ReadAllText(
14 datasetPath));
15     }
16
17     public JsonObject GetByIndex(int index)
18         => _data[index];
19 }
```

Podemos hacer algo similar para la colección de OutputResult

Boundries & Classes

Podemos aprovechar el encapsulamiento para manejar el calculo del accuracy

```
1 public class OutputResultCollection
2 {
3     private List<OutputResult> _results = new();
4     private int _successes = 0;
5
6     public void Add(OutputResult result)
7     {
8         if (result.ActualOutput == result.ExpectedOutput)
9         {
10             result.IsCorrect = true;
11             _successes++;
12         }
13         _results.Add(result);
14     }
15
16     public double GetAccuracy()
17     => (double)_successes / _results.Count;
18
19     public OutputResult[] ToArray()
20     => _results.ToArray();
21
22 }
```

Boundries & Classes

Podemos aprovechar el encapsulamiento para manejar el calculo del accuracy

```
1 public class OutputResultCollection
2 {
3     private List<OutputResult> _results = new();
4     private int _successes = 0;
5
6     public void Add(OutputResult result)
7     {
8         if (result.ActualOutput == result.ExpectedOutput)
9         {
10             result.IsCorrect = true;
11             _successes++;
12         }
13         _results.Add(result);
14     }
15
16     public double GetAccuracy()
17     => (double)_successes / _results.Count;
18
19     public OutputResult[] ToArray()
20     => _results.ToArray();
21
22 }
```

Luego, hay que modificar la evaluación para que aproveche las nuevas clases

Boundries & Classes

```
1 public abstract class EvaluationMethod(Dataset dataset)
2 {
3     public OutputResultCollection GenerateModelResults()
4     {
5         OutputResultCollection results = new();
6         for(int i = 0; i < dataset.Count; i++)
7         {
8             OutputResult result = GenerateInstanceResult(dataset.GetByIndex(i)
9         );
10            results.Add(result);
11        }
12        return results;
13    }
14
15    protected abstract OutputResult GenerateInstanceResult(JsonObject instance
16    );
17 }
```

Boundries & Classes

```
1 public abstract class EvaluationMethod(Dataset dataset)
2 {
3     public OutputResultCollection GenerateModelResults()
4     {
5         OutputResultCollection results = new();
6         for(int i = 0; i < dataset.Count; i++)
7         {
8             OutputResult result = GenerateInstanceResult(dataset.GetByIndex(i)
9         );
10            results.Add(result);
11        }
12        return results;
13    }
14
15    protected abstract OutputResult GenerateInstanceResult(JsonObject instance
16    );
17 }
```

Ahora solo falta cargar el OutputDir

Boundries & Classes

Esto no tiene ninguna ciencia, es solo copiar y pegar el código a otra clase

Boundries & Classes

Esto no tiene ninguna ciencia, es solo copiar y pegar el código a otra clase

```
1 public static class OutputWriter
2 {
3     public static void OutputToDirectory(OutputResultCollection results,
4     EvaluationRequest request)
5     {
6         if (request.OutputDir != null)
7         {
8             if (!Directory.Exists(request.OutputDir))
9                 throw new InvalidOutputDirectoryException(request.OutputDir);
10            var modelOutput = new ModelOutput(request.Model, request.Dataset,
11            request.EvalType, results.ToArray(),
12            results.GetAccuracy());
13            File.WriteAllText(Path.Combine(request.OutputDir!, "model_output.
14            json"),
15            JsonSerializer.Serialize(modelOutput, new
16            JsonSerializerOptions { WriteIndented = true }));
17        }
18    }
19 }
```

**¡Listo! Ya limpiamos
todo el código**

```
1 public class Evaluator {
2     private Result _result = new();
3
4     public Result Evaluate(EvaluationRequest request) {
5         try
6         {
7             _result = new Result();
8             TryToEvaluate(request);
9         }
10        catch (EvaluationException e)
11        {
12            _result.ErrorMessage = e.GetErrorMessage();
13            _result.Score = null;
14        }
15
16        return _result;
17    }
18
19    private void TryToEvaluate(EvaluationRequest request) {
20        tokenizer = new Tokenizer(request.Model);
21        Dataset dataset = new Dataset(request.Dataset);
22        IModel model = ModelFactory.Create(request.Model, tokenizer);
23        EvaluationMethod method = EvaluationMethodFactory.Create(request.EvalType, dataset, tokenizer,
24        model);
25        OutputResultCollection results = method.GenerateModelResults();
26        OutputWriter.OutputToDirectory(results, request);
27        BuildSuccessfulResult(results);
28    }
29
30    private void BuildSuccessfulResult(OutputResultCollection results) {
31        _result.Score = results.GetAccuracy();
32        _result.ErrorMessage = null;
33    }
34 }
```

Extra
Podemos limpiar un poco más

El modelo retorna un `double[]` con las probabilidades de generar cada palabra

```
1 public interface IModel {  
2     public double[] GenerateModelOutput(string input);  
3 }
```


El modelo retorna un `double[]` con las probabilidades de generar cada palabra

```
1 public interface IModel {  
2     public double[] GenerateModelOutput(string input);  
3 }
```

Luego usamos este `double[]` para calcular un ArgMax

```
1 private void GenerateOutput() {  
2     string auxInput = _result.Input;  
3     int[] encodedExpectedOutput = tokenizer.Encode(_result.ExpectedOutput);  
4     for (int j = 0; j < encodedExpectedOutput.Length; j++)  
5     {  
6         double[] probs = model.GenerateModelOutput(auxInput);  
7         int maxIdx = 0;  
8         for (int k = 1; k < probs.Length; k++)  
9         {  
10             if (probs[k] > probs[maxIdx])  
11                 maxIdx = k;  
12         }  
13         auxInput += tokenizer.Decode(maxIdx);  
14     }  
15     _result.ActualOutput = auxInput[_result.Input.Length..];  
16 }
```

El modelo retorna un `double[]` con las probabilidades de generar cada palabra

```
1 public interface IModel {  
2     public double[] GenerateModelOutput(string input);  
3 }
```

Luego usamos este `double[]` para calcular un ArgMax

```
1 private void GenerateOutput() {  
2     string auxInput = _result.Input;  
3     int[] encodedExpectedOutput = tokenizer.Encode(_result.ExpectedOutput);  
4     for (int j = 0; j < encodedExpectedOutput.Length; j++)  
5     {  
6         double[] probs = model.GenerateModelOutput(auxInput);  
7         int maxIdx = 0;  
8         for (int k = 1; k < probs.Length; k++)  
9         {  
10             if (probs[k] > probs[maxIdx])  
11                 maxIdx = k;  
12         }  
13         auxInput += tokenizer.Decode(maxIdx);  
14     }  
15     _result.ActualOutput = auxInput[_result.Input.Length..];  
16 }
```

Podemos encapsular este `double[]`

```
1 public class Probabilities
2 {
3     private double[] _probabilities;
4
5     public Probabilities(int index, Tokenizer tokenizer)
6     {
7         double[] probabilities =[.. Enumerable.Repeat(1.0, tokenizer.VocabularySize)];
8         probabilities[index] = tokenizer.VocabularySize;
9         double sum = probabilities.Sum();
10        _probabilities =[.. probabilities.Select(p => p / sum)];
11    }
12
13    public int ArgMax()
14    {
15        int maxIdx = 0;
16        for (int k = 1; k < _probabilities.Length; k++)
17        {
18            if (_probabilities[k] > _probabilities[maxIdx])
19                maxIdx = k;
20        }
21
22        return maxIdx;
23    }
24
25    public double GetByIndex(int index)
26        => _probabilities[index];
27 }
```

```
1 public class Probabilities
2 {
3     private double[] _probabilities;
4
5     public Probabilities(int index, Tokenizer tokenizer)
6     {
7         double[] probabilities =[.. Enumerable.Repeat(1.0, tokenizer.VocabularySize)];
8         probabilities[index] = tokenizer.VocabularySize;
9         double sum = probabilities.Sum();
10        _probabilities =[.. probabilities.Select(p => p / sum)];
11    }
12
13    public int ArgMax()
14    {
15        int maxIdx = 0;
16        for (int k = 1; k < _probabilities.Length; k++)
17        {
18            if (_probabilities[k] > _probabilities[maxIdx])
19                maxIdx = k;
20        }
21
22        return maxIdx;
23    }
24
25    public double GetByIndex(int index)
26        => _probabilities[index];
27 }
```

Ahora necesitamos que los modelos lo retornen

Tenemos que cambiar la interfaz IModel

```
1 public interface IModel
2 {
3     public Probabilities GenerateModelOutput(string input);
4 }
```

Tenemos que cambiar la interfaz IModel

```
1 public interface IModel
2 {
3     public Probabilities GenerateModelOutput(string input);
4 }
```

Nuestras clases de modelo se verán así:

```
1 public class RandomModel(Tokenizer tokenizer) : IModel
2 {
3     private int _randomCount = 0;
4
5     public Probabilities GenerateModelOutput(string input)
6     {
7         int randomIndex = _randomCount % tokenizer.VocabularySize;
8         Probabilities probabilities = new Probabilities(randomIndex, tokenizer
9     );
10         _randomCount++;
11         return probabilities;
12     }
```

Finalmente, en los lugares donde usamos el modelo, utilizaremos el método ArgMax()

```
1 private void GenerateOutput()  
2 {  
3     string auxInput = _result.Input;  
4     int[] encodedExpectedOutput = tokenizer.Encode(_result.ExpectedOutput);  
5     for (int j = 0; j < encodedExpectedOutput.Length; j++)  
6     {  
7         Probabilities probs = model.GenerateModelOutput(auxInput);  
8         int maxIdx = probs.ArgMax();  
9         auxInput += tokenizer.Decode(maxIdx);  
10    }  
11    _result.ActualOutput = auxInput[_result.Input.Length..];  
12 }
```

Finalmente, en los lugares donde usamos el modelo, utilizaremos el método ArgMax()

```
1 private void GenerateOutput()  
2 {  
3     string auxInput = _result.Input;  
4     int[] encodedExpectedOutput = tokenizer.Encode(_result.ExpectedOutput);  
5     for (int j = 0; j < encodedExpectedOutput.Length; j++)  
6     {  
7         Probabilities probs = model.GenerateModelOutput(auxInput);  
8         int maxIdx = probs.ArgMax();  
9         auxInput += tokenizer.Decode(maxIdx);  
10    }  
11    _result.ActualOutput = auxInput[_result.Input.Length..];  
12 }
```

¡Listo! Ya tenemos nuestros modelos un poco más limpios